



Computer Networks CL3001

LAB - 07 NS3

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES,
KARACHI CAMPUS
FAST SCHOOL OF COMPUTING (AI & DS, CS, CY, SE)
SPRING 2025



Computer Networks Lab 07

Course: Computer Networks (CL3001)

Semester: Spring 2025

Instructor: Muhammad Nadeem Ghouri

Note:

- Maintain discipline during the lab.
- Listen and follow the instructions as they are given.
- Just raise hand if you have any problem.
- Completing all tasks of each lab is compulsory.
- Get your lab checked at the end of the session.

Lab Objective

- Introduction to NS3 network simulator.
- Simple network simulation in NS3.
- Implementation of TCP congestion control & simulation in NS3.

NS3

1. Introduction

NS3 is a discrete event simulator targeted at networking research. NS3 provides substantial support for simulation of TCP, UDP, routing & multicast protocols over wired & wireless (local & satellite) networks. This simulator is primarily UNIX based & the NS commands can either be entered via UNIX command prompt or by running a scripting language file, but scripting language is preferred way. NS3 uses Tcl (pronounced as 'tickle') as its scripting language, which is edited using a text editor & in this example we will use vi, gedit or nano.

2. Introduction to vi

vi is a text editor that is available in all UNIX systems, other editors such as pico & emacs can also be used instead. Before you start for the first time, you must note that vi is a modal editor. A mode is like an environment. Different modes in vi interpret the same key differently. For example, if you're in insert mode, typing 'a' adds an 'a' to the text, whereas in command mode, typing 'a' puts you in insert mode because 'a' is the key abbreviation for the append command. If you get confused about what mode you're in, press the Escape key. Pressing the Escape key always returns you to the command mode & if you are already in command mode, it simply beeps to remind you of the fact.

When you are in command mode, you can manage your document; this includes the capability to change text, rearrange it, & delete it. Insert mode is when you maneuver the text area using the arrow keys.



For starting a new document, simply type `vi` after the command prompt to start the `vi` editor. The cursor will be located in the top left corner & each of the following lines will start with a tilde (~) denoting empty lines. Note that the `vi` editor is currently in command mode. In order to edit an existing file, type the name of the file along with the extension after typing `vi` & a space in the UNIX command prompt. To enter into insert mode simply press `a` on the keyboard. Then to save the edited document, type a colon (`:`) & the cursor will be located in the bottom left corner after the colon, followed by a `w` & then press enter. In order to quit `vi` editor type a colon followed by a `q` (`:q`), this will take you back to the UNIX prompt. You can also choose to quit editing without saving by typing `:q!`, & if you forget the exclamation mark (!) the system will issue a warning.

Note: direct link for NS 3 image:

<https://www.nsnam.com/2020/04/download-vm-image-of-ns3-and-contiking.html>

3. Simple Simulation in NS3

This section shows a simple `ns` simulation script and explains what each line does. Example 1 is an `otcl` script that creates the simple network configuration and runs the simulation scenario in figure 1. To run this simulation, download "`ns-simple.tcl`" and type "`ns ns-simple.tcl`" at your shell prompt.

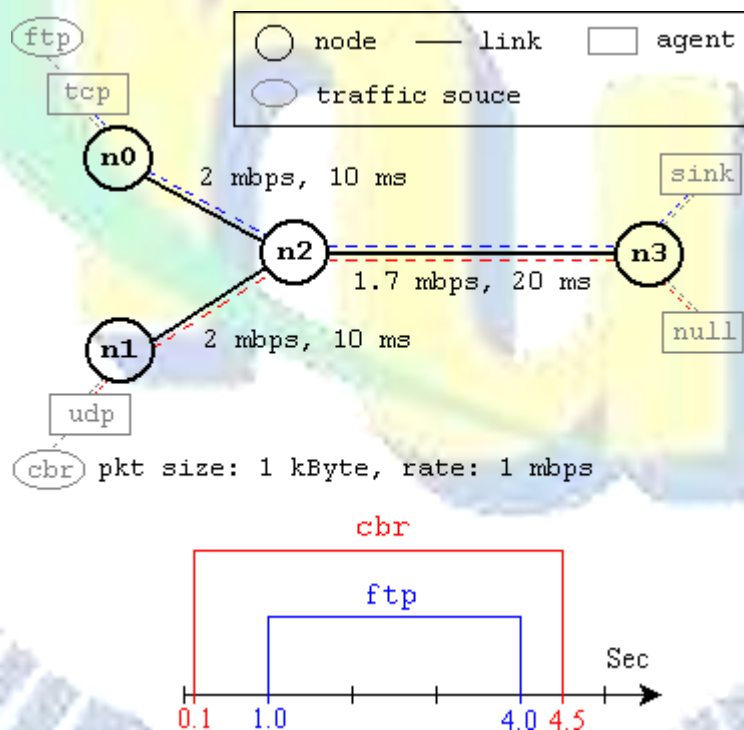


Fig-1: A simple network topology & simple simulation scenario

This network consists of 4 nodes (n0, n1, n2, n3) as shown in above figure. The duplex links between n0 and n2, and n1 and n2 have 2 Mbps of bandwidth and 10 ms of delay. The duplex link between n2 and n3 has 1.7 Mbps of bandwidth and 20 ms of delay. Each node uses a DropTail queue, of which the maximum size is 10. A "tcp" agent is attached to n0, and a connection is established to a tcp "sink" agent



attached to n3. As default, the maximum size of a packet that a "tcp" agent can generate is 1KByte. A tcp "sink" agent generates and sends ACK packets to the sender (tcp agent) and frees the received packets. A "udp" agent that is attached to n1 is connected to a "null" agent attached to n3. A "null" agent just frees the packets received. A "ftp" and a "cbr" traffic generator are attached to "tcp" and "udp" agents respectively, and the "cbr" is configured to generate 1 KByte packets at the rate of 1 Mbps. The "cbr" is set to start at 0.1 sec and stop at 4.5 sec, and "ftp" is set to start at 1.0 sec and stop at 4.0 sec.

```
#Create a simulator objectset ns [new Simulator]

#Define different colors for data flows (for NAM)

$ns color 1 Blue
$ns color 2 Red

#Open the NAM trace fileset nf [open out.nam w]

$ns namtrace-all $nf

#Define a 'finish' procedureproc finish {} {
    global ns nf
    $ns flush-trace
    #Close the NAM trace fileclose $nf
    #Execute NAM on the trace fileexec nam out.nam &
    exit 0
}

#Create four nodesset n0 [$ns node] set n1 [$ns node]
    set n2 [$ns node] set n3 [$ns node]

#Create links between the nodes

$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns duplex-link $n2 $n3 1.7Mb 20ms DropTail

#Set Queue Size of link (n2-n3) to 10

$ns queue-limit $n2 $n3 10
```



```
#Give node position (for NAM)
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right

#Monitor the queue for link (n2-n3). (for NAM)
$ns duplex-link-op $n2 $n3 queuePos 0.5

#Setup a TCP connectionset tcp [new Agent/TCP]
$tcp set class_ 2
$ns attach-agent $n0 $tcp

set sink [new Agent/TCPSink]

$ns attach-agent $n3 $sink
$ns connect $tcp $sink
$tcp set fid_ 1

#Setup a FTP over TCP connectionset ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP

#Setup a UDP connectionset udp [new Agent/UDP]
$ns attach-agent $n1 $udpset null [new Agent/Null]
$ns attach-agent $n3 $null
$ns connect $udp $null
$udp set fid_ 2

#Setup a CBR over UDP connection set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 1mb
$cbr set random_ false

#Schedule events for the CBR and FTP agents
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 4.0 "$ftp stop"
$ns at 4.5 "$cbr stop"
```



```
#Detach tcp and sink agents (not really necessary)
$ns at 4.5 "$ns detach-agent $n0 $tcp ; $ns detach-agent $n3 $sink"

#Call the finish procedure after 5 seconds of simulation time
$ns at 5.0 "finish"

#Print CBR packet size and interval
puts "CBR packet size = [$cbr set packet_size_]" puts "CBR interval = [$cbr set interval_]"

#Run the simulation
$ns run
```

Example 1: A Simple NS Simulation Script

4. Code Explanation

The following is the explanation of the script above. In general, an NS script starts with making a Simulator object instance.

set ns [new Simulator]: generates an NS simulator object instance, and assigns it to variable *ns* (italics is used for variables and values in this section). What this line does is the following:

- Initialize the packet format (ignore this for now)
- Create a scheduler (default is calendar scheduler)
- Select the default address format (ignore this for now)

The "Simulator" object has member functions that do the following:

- Create compound objects such as nodes and links (described later)
- Connect network component objects created (ex. attach-agent)
- Set network component parameters (mostly for compound objects)
- Create connections between agents (ex. make connection between a "tcp" and "sink")
- Specify NAM display options
- Etc.

Most of member functions are for simulation setup (referred to as plumbing functions) and scheduling, however some of them are for the NAM display. The "Simulator" object memberfunction implementations are located in the "ns-2/tcl/lib/ns-lib.tcl" file.

\$ns color fid color: is to set color of the packets for a flow specified by the flow id (fid). This member function of "Simulator" object is for the NAM display, and has no effect on the actualsimulation.

\$ns namtrace-all file-descriptor: This member function tells the simulator to record simulation traces in NAM input format. It also gives the file name that the trace will be written to later by thecommand



\$ns flush-trace. Similarly, the member function trace-all is for recording the simulation trace in a general format.

proc finish {}: is called after this simulation is over by the command **\$ns at 5.0 "finish"**. In this function, post-simulation processes are specified.

set n0 [\$ns node]: The member function **node** creates a node. A node in NS is compound object made of address and port classifiers (described in a later section). Users can create a node by separately creating an address and a port classifier object and connecting them together. However, this member function of Simulator object makes the job easier. To see how a node is created, look at the files: "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-node.tcl".

\$ns duplex-link node1 node2 bandwidth delay queue-type: creates two simplex links of specified bandwidth and delay, and connects the two specified nodes. In NS, the output queue of a node is implemented as a part of a link, therefore users should specify the queue-type when creating links. In the above simulation script, DropTail queue is used. If the reader wants to use a RED queue, simply replace the word DropTail with RED. The NS implementation of a link is shown in a later section. Like a node, a link is a compound object, and users can create its sub-objects and connect them and the nodes. Link source codes can be found in "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-link.tcl" files. One thing to note is that you can insert error modules in a link component to simulate a lossy link (actually users can make and insert any network objects). Refer to the NS documentation to find out how to do this.

\$ns queue-limit node1 node2 number: This line sets the queue limit of the two simplex links that connect node1 and node2 to the number specified. At this point, the authors do not know how many of these kinds of member functions of Simulator objects are available and what they are. Please take a look at "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-link.tcl", or NS documentation for more information.

\$ns duplex-link-op node1 node2 ...: The next couple of lines are used for the NAM display. To see the effects of these lines, users can comment these lines out and try the simulation.

Now that the basic network setup is done, the next thing to do is to setup traffic agents such as TCP and UDP, traffic sources such as FTP and CBR, and attach them to nodes and agents respectively.

set tcp [new Agent/TCP]: This line shows how to create a TCP agent. But in general, users can create any agent or traffic sources in this way. Agents and traffic sources are in fact basic objects (not compound objects), mostly implemented in C++ and linked to OTcl. Therefore, there are no specific Simulator object member functions that create these object instances. To create agents or traffic sources, a user should know the class names these objects (Agent/TCP, Agent/TCPSink, Application/FTP and so on). This information can be found in the NS documentation or partly in this documentation. But one shortcut is to look at the "ns-2/tcl/libs/ns-default.tcl" file. This file contains the default configurable parameter value settings for available network objects. Therefore, it works as a good indicator of what kind of network objects are available in NS and what are the configurable parameters.

\$ns attach-agent node agent: The **attach-agent** member function attaches an agent object created to a node object. Actually, what this function does is call the **attach** member function of specified node, which attaches the given agent to itself. Therefore, a user can do the same thing by, for example, **\$n0 attach \$tcp**. Similarly, each agent object has a member function **attach-agent** that attaches a traffic source object to itself.



\$ns connect agent1 agent2: After two agents that will communicate with each other are created, the next thing is to establish a logical network connection between them. This line establishes a network connection by setting the destination address to each other's network and port address pair.

Assuming that all the network configuration is done, the next thing to do is write a simulation scenario (i.e. simulation scheduling). The Simulator object has many scheduling members functions. However, the one that is mostly used is the following:

\$ns at time "string": This member function of a Simulator object makes the scheduler (scheduler_ is the variable that points the scheduler object created by [new Scheduler] command at the beginning of the script) to schedule the execution of the specified string at given simulation time. For example, **\$ns at 0.1 "\$cbr start"** will make the scheduler call a **start** member function of the CBR traffic source object, which starts the CBR to transmit data. In NS, usually a traffic source does not transmit actual data, but it notifies the underlying agent that it has some amount of data to transmit, and the agent, just knowing how much of the data to transfer, creates packets and sends them.

After all network configuration, scheduling and post-simulation procedure specifications are done, the only thing left is to run the simulation. This is done by **\$ns run**.

The above example and information come from the [Worcester Polytechnic Institute](#) and from the site [Nsby Example](#). There you can find many examples for Ns Simulator and additional resources.

5. TCP Congestion Control Simulation

In this example we will implement & simulate congestion control mechanism of TCP between two nodes. It consists of three parts which are as follow.

1. Slow start threshold (exponential increase of packets) ssthresh
2. Congestion avoidance (additive increase)
3. Congestion detection (multiplicative decrease)

First.cc

```
/* -*- Mode:C++; c-file-style:"gnu"; indent-tabs-mode:nil; -*- */
/*
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License version 2 as
 * published by the Free Software Foundation;
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software
```




*** Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA
*/**

```
#include "ns3/core-module.h"  
#include "ns3/network-module.h"  
#include "ns3/internet-module.h"  
#include "ns3/point-to-point-module.h"  
#include "ns3/applications-module.h"  
  
using namespace ns3;  
  
NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");  
  
int main (int argc, char *argv[])  
{  
    CommandLine cmd;  
  
    cmd.Parse (argc, argv);  
  
    Time::SetResolution (Time::NS);  
  
    LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);  
    LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);  
  
    NodeContainer nodes;  
  
    nodes.Create (2);  
  
    PointToPointHelper pointToPoint;  
    pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));  
    pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));  
  
    NetDeviceContainer devices;  
  
    devices = pointToPoint.Install (nodes);  
  
    InternetStackHelper stack;  
    stack.Install (nodes);  
  
    Ipv4AddressHelper address;  
    address.SetBase ("10.1.1.0", "255.255.255.0");
```



```
Ipv4InterfaceContainer interfaces = address.Assign (devices);

UdpEchoServerHelper echoServer (9);

ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));

serverApps.Start (Seconds (1.0));

serverApps.Stop (Seconds (10.0))

UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);

echoClient.SetAttribute ("MaxPackets", UIntegerValue (1));

echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));

echoClient.SetAttribute ("PacketSize", UIntegerValue (1024));

ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));

clientApps.Start (Seconds (2.0));

clientApps.Stop (Seconds (10.0));

Simulator::Run ();

Simulator::Destroy ();

return 0;
}
```

Second.cc

```
/* -*- Mode:C++; c-file-style:"gnu"; indent-tabs-mode:nil; -*- */
/*
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License version 2 as
 * published by the Free Software Foundation;
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
```



*** GNU General Public License for more details.**

*** You should have received a copy of the GNU General Public License**

*** along with this program; if not, write to the Free Software**

*** Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA**

***/**

#include "ns3/core-module.h"

#include "ns3/network-module.h"

#include "ns3/csma-module.h"

#include "ns3/internet-module.h"

#include "ns3/point-to-point-module.h"

#include "ns3/applications-module.h"

#include "ns3/ipv4-global-routing-helper.h"

#include "ns3/netanim-module.h"

// Default Network Topology

//

// 10.1.1.0

// n0 ----- n1 n2 n3 n4

// point-to-point |||

// =====

// LAN 10.1.2.0

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");

int main (int argc, char *argv[])

{

bool verbose = true;

uint32_t nCsma = 3;

CommandLine cmd;

cmd.AddValue ("nCsma", "Number of \"extra\" CSMA nodes/devices", nCsma);

cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);

cmd.Parse (argc,argv);

if (verbose)

{

LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);

LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);

}

nCsma = nCsma == 0 ? 1 : nCsma;

NodeContainer p2pNodes;



```
p2pNodes.Create (2);
NodeContainer csmaNodes;
csmaNodes.Add (p2pNodes.Get (1));
csmaNodes.Create (nCsmas);
PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
NetDeviceContainer p2pDevices;
p2pDevices = pointToPoint.Install (p2pNodes);
CsmaHelper csma;
csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));
csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));
NetDeviceContainer csmaDevices;
csmaDevices = csma.Install (csmaNodes);

InternetStackHelper stack;
stack.Install (p2pNodes.Get (0));
stack.Install (csmaNodes);
Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer p2pInterfaces;
p2pInterfaces = address.Assign (p2pDevices);
address.SetBase ("10.1.2.0", "255.255.255.0");
Ipv4InterfaceContainer csmaInterfaces;
csmaInterfaces = address.Assign (csmaDevices);
UdpEchoServerHelper echoServer (9);
ApplicationContainer serverApps = echoServer.Install (csmaNodes.Get (nCsmas));
serverApps.Start (Seconds (1.0));
serverApps.Stop (Seconds (10.0));
UdpEchoClientHelper echoClient (csmaInterfaces.GetAddress (nCsmas), 9);
echoClient.SetAttribute ("MaxPackets", UIntegerValue (1));
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient.SetAttribute ("PacketSize", UIntegerValue (1024));
ApplicationContainer clientApps = echoClient.Install (p2pNodes.Get (0));
clientApps.Start (Seconds (2.0));
clientApps.Stop (Seconds (10.0));
Ipv4GlobalRoutingHelper::PopulateRoutingTables ();
pointToPoint.EnablePcapAll ("second");
csma.EnablePcap ("second", csmaDevices.Get (1), true);

AnimationInterface anim ("second.xml");
Simulator::Run ();
Simulator::Destroy ();
return 0;
}
```



6. Code Output

```
File Edit View Search Terminal Help
7.95312 9801
7.96144 9830
7.96976 9859
7.97808 9888
7.9864 9917
RxDrop at 7.99174
8.00306 1072
8.01306 1340
8.01968 1554
8.028 1738
8.03632 1903
8.04464 2053
8.05296 2192
```

Fig-2: Output

After running the code & getting above output now see generated files and use gnuplot to display the graphs.

```
pradeepkumar@pradeepkumar:~/ns-allinone-3.27/ns-3.27$ ls
AUTHORS      examples      seventh.cwnd      seventh.pcap      utils.pyc      wutils.py
bindings     LICENSE       seventh-packet-byte-count-0.txt  src               VERSION        wutils.pyc
build        Makefile      seventh-packet-byte-count-1.txt  test.py           waf
CHANGES.html  README       seventh-packet-byte-count.dat    testpy.supp       waf.bat
contrib       RELEASE_NOTES seventh-packet-byte-count.plt     utils             waf-tools
doc           scratch      seventh-packet-byte-count.sh     utils.py          wscript
pradeepkumar@pradeepkumar:~/ns-allinone-3.27/ns-3.27$ gnuplot seventh-packet-byte-count.plt
```

Fig-3: List of generated files and commands to use gnuplot

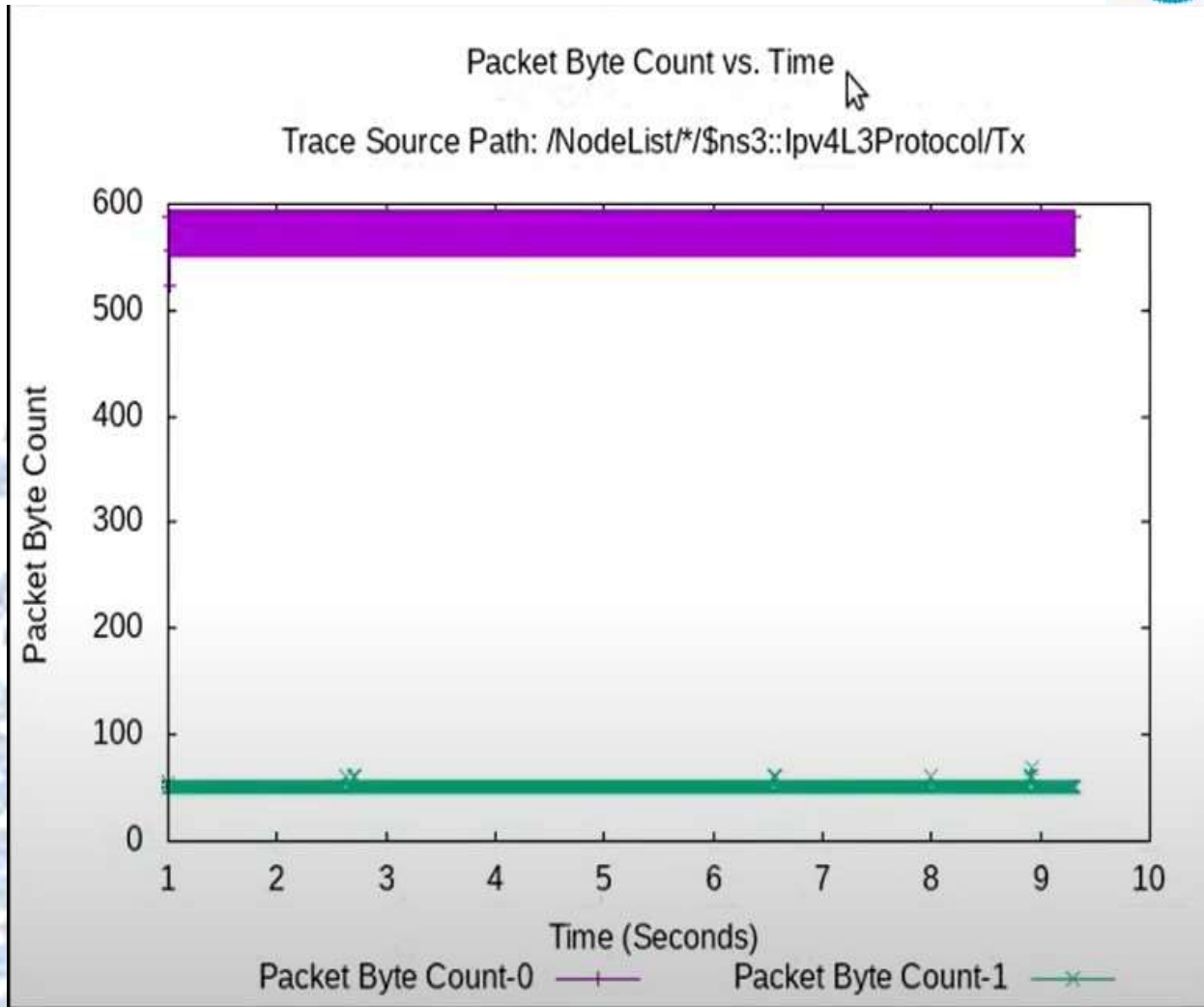


Fig-4: Packet byte count vs time graph

To plot congestion control graph first open text editor in UNIX and type the following script and save the file using **.plt** extension. Then use type the command on terminal shown in figure 6.

```
set terminal png
set output "congestion.png"
set title "Congestion Window Calculation"
set xlabel "Time (in Seconds)"
set ylabel "Congestion Window (cwnd)"
plot "seventh.cwnd" using 1:2 with linespoints title "Old Congestion", "seventh.cwnd"
using 1:3 with linespoints title "New Congestion"
```

Fig-5: Congestion graph plot script

```
:~/ns-allinone-3.27/ns-3.27$ gnuplot congestion.plt
```

Fig-6: Command to plot congestion control graph

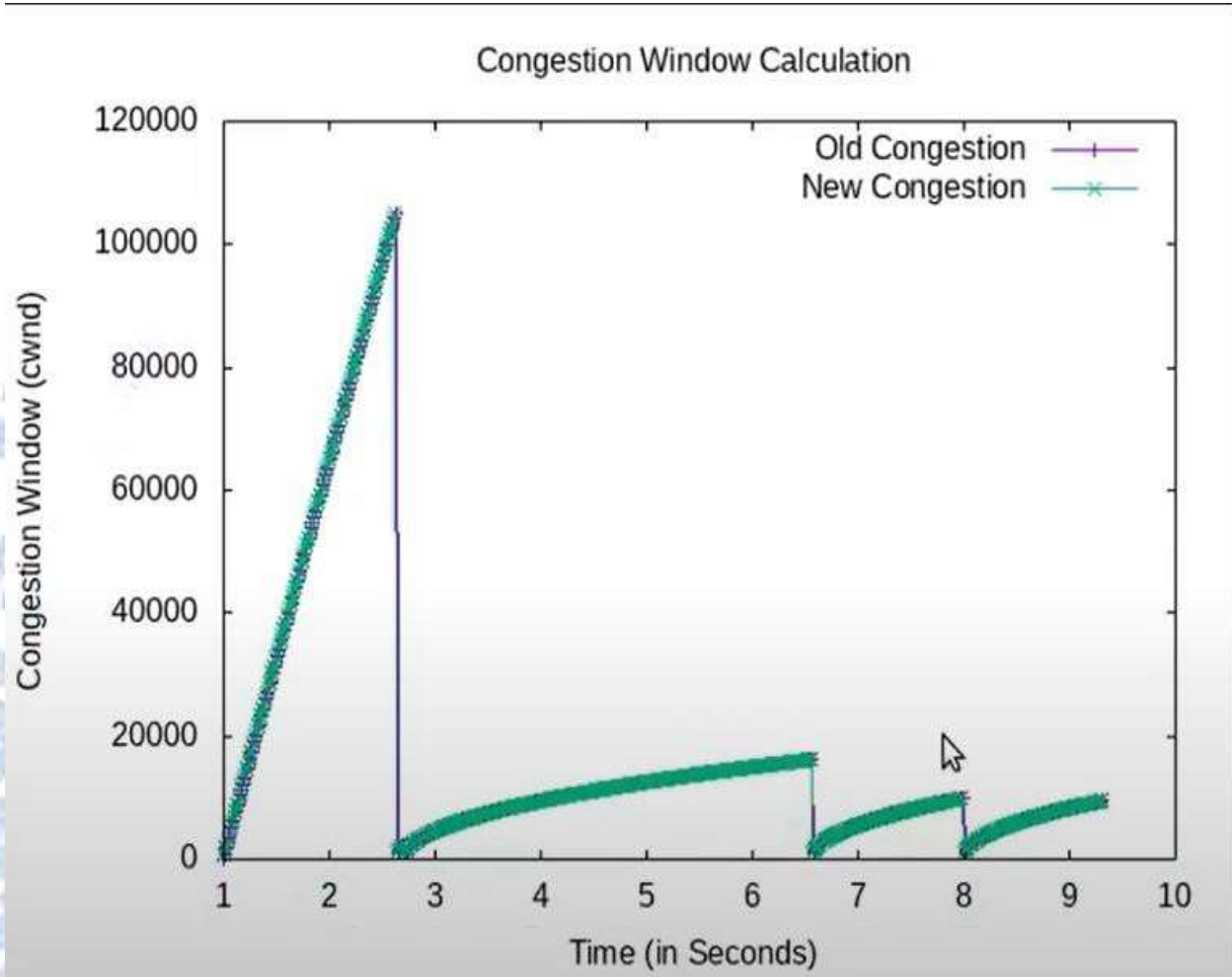


Fig-7: Output Graph

7. Bandwidth Analysis

buffer_management.cc



Lab 07 Tasks Report Submission

Lab Task

1. Apply TCP Congestion control mechanism in NS3 & plot the congestion window graph.

Helpful link for TCP congestion Task:

<https://www.youtube.com/watch?v=9rkN3FtOkaQ&t=885s>

2. Create a `bandwidth_analysis.cc` file to demonstrate different bandwidth values affecting packet delivery and throughput. Create a point-to-point network with two nodes and set up varying bandwidths (1, 5, 10) Mbps, then generate UDP traffic and measure throughput and packet loss using flow monitor and finally run simulations with different bandwidth values.
3. Create a `buffer_management.cc` file to demonstrate different queue sizes are impacting on packet drop ratio. Create a point-to-point network with a 2 Mbps link and set up varying queue sizes (5, 10, 50) packets, then generate UDP traffic with higher transmission rate 5 Mbps to create congestion and to monitor packet loss statistics use flow monitor and finally compare packet loss for different queue sizes.
4. Create a `bottleneck_analysis.cc` file to demonstrate bottleneck where multiple senders overload a limited bandwidth link. Create a network topology which includes 4 sender nodes, 2 routers, and 4 receiver nodes and configure bottleneck link between routers with only 1Mbps bandwidth. Sender generates UDP traffic to receiver and uses flow monitor to measure throughput per flow. Also increase traffic load and analyze its impact.
5. Create a `data_loss.cc` file to demonstrate data loss in Wifi network under different conditions. Create a Wifi network which includes 1 Access Point (AP) and 3 Stations, then generate UDP traffic from AP to Station and vary distances between AP and Station (10, 50, 100) m and introduce interference by additional traffic from different stations and use flow monitor to measure packet loss and signal strength.